TREE TRAVERSALS

"Four ways to visit every node — three recursive twins and one queue-based outsider."

▪ FROM COUNTING TO VISITING ▪

Last class we counted nodes and found the max. Today we visit every single node, in a specific order, and do something at each visit. That's a **traversal**. Four flavors — three recursive cousins and one queue-based outsider.

ID	Concept & Mechanics
[01]	**Preorder, inorder, postorder** Three DFS traversals. Same skeleton. One line moves.
[02]	**BFS — breadth-first** Visits level by level. Uses a queue, not recursion.
[03]	**The reflex of the day** "Do I need parent BEFORE, BETWEEN, or AFTER the children?"

"Same skeleton, different action. THE WALK pattern, one more time."

▪ FOUR ORDERS, FOUR USE CASES ▪

Different orders solve different problems. Pick the wrong order and the algorithm collapses. Here's the cheat sheet — keep it open while you code.

Traversal	When to reach for it
Preorder	Save a tree to a file (so you can rebuild it later)
Inorder	Read a Binary Search Tree's values in **sorted order** (Class 07!)
Postorder	Compute the size of every folder before its parent (think `du`)
BFS	Find the shortest path in an unweighted graph; render level by level

"The order is not decoration. It decides what the algorithm can do."

THE THREE TWINS

for the three transversal, the skeleton is identical.

The only thing that changes is **which line holds the visit**. Once you've written one, you've written all three.

```
def traversal(node):
if node is None:
return

# ← preorder VISITS HERE
traversal(node.left)
# ← inorder VISITS HERE
traversal(node.right)
# ← postorder VISITS HERE
```

Same skeleton.

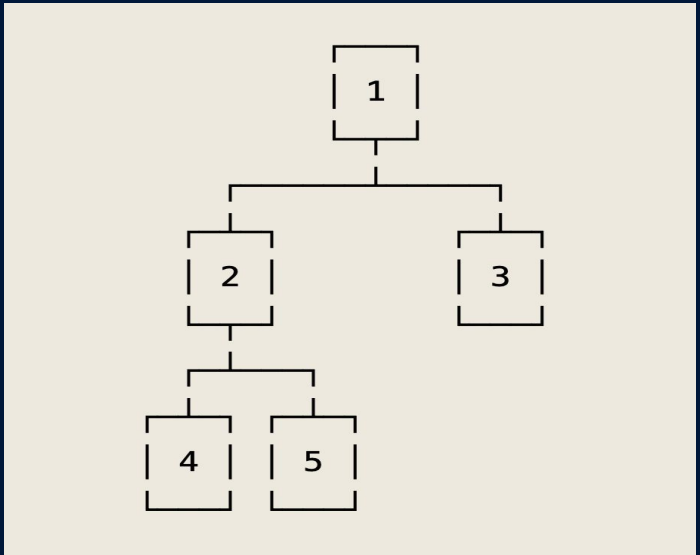
One line moves.

That is literally the only difference between the three DFS traversals.

"Three traversals for the price of one. Move the visit, change the order."

ONE TREE, FOUR ORDERS

Keep this tree in your head for the rest of the class. Every traversal below visits these five nodes — but in a different order. The numbers make the orders easy to compare.



Five nodes.
Four traversals.
Five values printed each time, in five different orders.

"Memorize this tree. We'll trace four traversals on it."

▪ DEPTH-FIRST SEARCH ▪

Depth-First means: when you reach a node, you go DEEP into one branch before looking at the others. You only come back up after a whole subtree is done.

At each node you do three things — visit, go left, go right. The DFS family is just three different orders for those three actions.

Three actions per node — VISIT, RECURSE LEFT, RECURSE RIGHT. The DFS family is three orderings of these three lines.

[01] Preorder
Visit BEFORE descending. Root comes first.

[02] Inorder
Visit BETWEEN left and right. The middle slot.

[03] Postorder
Visit AFTER descending. Root comes last.

"Three actions, three orderings. That's the whole DFS family."

▪ PREORDER ▪

Order: **root → left → right**. We print BEFORE going down.

So the root comes first, then everything in the left subtree (still root-first there), then everything in the right.

```
def preorder(node):
if node is None: # base case
return

print(node.value) # 1) VISIT current
preorder(node.left) # 2) recurse LEFT
preorder(node.right) # 3) recurse RIGHT
```

[OUTPUT] On reference tree:

1, 2, 4, 5, 3

Use cases:

- Serializing a tree
- Copying a tree
- Top-down exploration

"Visit first, descend after. Root always opens the show."

INORDER

Order: **left → root → right**. We descend left first, then visit, then go right.

The visit slot lives in the middle of the function — same skeleton as preorder, one line moved.

```
def inorder(node):
if node is None:
return

inorder(node.left) # 1) recurse LEFT first
print(node.value) # 2) VISIT current
inorder(node.right) # 3) recurse RIGHT
```

Killer use case (next class!) — on a Binary Search Tree, inorder gives values in sorted order.

Output here:

4, 2, 5, 1, 3

"Inorder + BST = sorted output. The punchline of Class 07."

▪ POSTORDER ▪

Order: **left → right → root**. We descend both subtrees first, and visit only at the very end.

The root is visited LAST. Useful when you need to know everything about the children before computing the parent.

```
def postorder(node):
if node is None:
return

postorder(node.left) # 1) recurse LEFT
postorder(node.right) # 2) recurse RIGHT
print(node.value) # 3) VISIT current LAST
```

On our reference tree:

4, 5, 2, 3, 1

Perfect for:

- Folder sizes (bottom-up)
- Tree heights
- Freeing memory

"Children first, parent last. Postorder = children-before-parents."

DFS USES A STACK. BFS NEEDS A QUEUE.

DFS works naturally with recursion because Python's call stack is itself a stack — perfect for "go deep, then come back".

BFS needs the opposite behavior: process the **oldest** pending node first, not the newest. That is the definition of a queue (FIFO) — and it's why we need the `deque` from Class 02.

❌ Recursion / stack — what DFS uses

```
visit(node.left) # newest call first
visit(node.right)
```

LIFO — last call in is the first one out. Goes **DEEP**.

✅ Queue / deque — what BFS needs

```
queue.append(node.right)
oldest = queue.popleft() # oldest waits
```

FIFO — first in is the first out. Goes **WIDE**.

"Recursion gives you depth for free. For breadth, you bring your own queue."

BREADTH-FIRST SEARCH WITH DEQUE

Start with the root in the queue. Loop: take the oldest node out, visit it, append its children to the end. Repeat until the queue is empty.

Always `popleft()`, always `append()` — that's the FIFO discipline.

```python
from collections import deque

def bfs(root):
    if root is None:
        return

    queue = deque([root]) # start with root
    while queue:
        node = queue.popleft() # OLDEST out
        print(node.value)
        if node.left is not None:
            queue.append(node.left)
        if node.right is not None:
            queue.append(node.right)
```

[01] Initialization
The deque is initialized with the root node to kick off the BFS process.

[02] Level Order
By popping from the left and appending to the right, we process nodes in the order they were discovered.

[03] Termination
The loop continues until no more reachable nodes are waiting in the queue.

"popleft + append = FIFO. That single discipline gives you BFS."

BFS LEVEL BY LEVEL

Watch the queue evolve. Each step pops the front, visits it, and appends the children at the back. The queue grows to a "front" of unvisited nodes at the same depth, then shrinks as it eats them in order.

```
queue: [1] pop 1, visit 1, enqueue children → [2, 3]
queue: [2, 3] pop 2, visit 2, enqueue children → [3, 4, 5]
queue: [3, 4, 5] pop 3, visit 3, no children → [4, 5]
queue: [4, 5] pop 4, visit 4, no children → [5]
queue: [5] pop 5, visit 5, no children → []
queue: [] loop ends
```

Output: 1, 2, 3, 4, 5 — exactly level by level

Output: 1, 2, 3, 4, 5

Root, then level 1, then level 2. BFS visits by **distance from the root**.

"BFS visits by distance. The first time you see a node, you saw it via the shortest path."

▪ THE REFLEX ▪

A small mental table to keep next to you when you face a tree problem. Read the question on the left, get the traversal on the right. If nothing matches, default to preorder.

Question to ask	Use
Do I need the parent BEFORE the children?	**Preorder**
Do I need the values in sorted order (BST)?	**Inorder**
Do I need the children BEFORE the parent? (folder size)	**Postorder**
Do I need to visit by distance / level by level?	**BFS**

When in doubt, try preorder first. It's the most natural top-down exploration and works for most tasks.

"Parent before, between, after, or by-level. Pick one — that's the whole game."